Laboration 2 – Rapport

Uppgift 1 – Generics

1. Varför kompilerar inte

boxTest(new Box<Integer>);

Denna kod kompilerar inte eftersom man saknar parenteser vid skapandet av objektet. I Java måste man alltid anropa konstruktorn med parenteser, även om den inte tar några argument.

Korrekt kod är:

boxTest(new Box<Integer>());

Utan parenteserna försöker man referera till klassen istället för att skapa ett objekt, vilket leder till kompileringsfel.

2. Förklaring av

public class BinarySearchTree<T extends Comparable<? super T>>

Denna deklaration består av flera delar:

- **T**: En generisk typ som gör klassen flexibel så att den kan hantera olika datatyper.
- **extends Comparable**: Detta innebär att typen T måste implementera interfacet Comparable. Detta krävs för att kunna jämföra element i trädet (t.ex. för att avgöra om ett värde ska till vänster eller höger).
- **<? super T>**: Detta innebär att T kan jämföras med sig själv eller med en superklass till T.

Anledningen till denna deklaration är att ett binärt sökträd kräver att elementen kan jämföras för att kunna placeras korrekt i trädet.

3. Vad innebär PECS Rule?

PECS står för:

Producer Extends, Consumer Super

Regeln används vid användning av generics med wildcards:

- **Producer Extends (? extends T)** används när man endast läser data från en struktur.
- **Consumer Super (? super T)** används när man skriver/lägger till data i en struktur.

Detta hjälper till att undvika typfel och gör koden mer flexibel och säker.

Uppgift 2 – Binära sökträd

Beskrivning av implementationen

I denna laboration har ett binärt sökträd (Binary Search Tree, BST) implementerats i Java. Trädet består av noder som är sammanlänkade med referenser. Varje nod innehåller ett värde samt referenser till sina vänstra och högra barn.

Klassen `BinarySearchTree<T extends Comparable<? super T>>` använder generics för att kunna hantera olika typer av data, så länge de kan jämföras. Detta är nödvändigt eftersom ett BST bygger på att element kan jämföras för att placeras korrekt i trädet.

En inre klass `BSTNode` används för att representera varje nod i trädet. Den innehåller:

- ett värde av typen `T`
- en referens till vänster barn
- en referens till höger barn

Själva trädet har två attribut:

- `root` – referens till rot-noden
- `size` – antal element i trädet

Trädet följer BST-regeln:

- Alla värden i vänstra delträdet är mindre än nodens värde
- Alla värden i högra delträdet är större än nodens värde

Metoden add(T item)

Metoden `add` används för att lägga till ett nytt element i trädet. Elementet placeras på rätt position enligt BST-regeln.

I denna implementation hanteras dubbletter genom att de **ignoreras**, vilket innebär att om ett element redan finns i trädet så läggs det inte till igen.

Pseudokod:

```
public class pesudokod {
//  add(item):
//      root = addRecursive(root, item)
//
//  addRecursive(node, item):
//      if node == null:
//          skapa ny nod med item
//          öka size
//          return nod
//
//      if item < node.item:
//          node.left = addRecursive(node.left, item)
//
//      else if item > node.item:
//          node.right = addRecursive(node.right, item)
//
//      annars (item == node.item):
//          gör ingenting (ignorera dubblett)
//
//      return node

}
```

Basfall och rekursion

- **Basfall:** När aktuell nod är `null` → ny nod skapas
- **Rekursionssteg:** Metoden anropas på vänster eller höger barn beroende på jämförelse

Metoden remove(T itemToRemove)

Beskrivning

Metoden `remove` används för att ta bort ett element från trädet. Om elementet finns tas det bort och metoden returnerar `true`, annars returneras `false`.

Att ta bort en nod i ett BST är mer komplext än att lägga till och består av tre fall:

1. **Noden är ett löv (inga barn)** → tas bort direkt
2. **Noden har ett barn** → barnet ersätter noden
3. **Noden har två barn** → ersätt nodens värde med minsta värdet i höger delträd och ta bort den noden

Pseudokod:

```
public class pesudokod {
//  remove(item):
//      root = removeRecursive(root, item)
//
//  removeRecursive(node, item):
//      if node == null:
//          return null
//
//      if item < node.item:
//          node.left = removeRecursive(node.left, item)
//
//      else if item > node.item:
//          node.right = removeRecursive(node.right, item)
//
//      else:
//          // noden hittad
//
//          if node har inga barn:
//              return null
//
//          if node har ett barn:
//              return barnet
//
//          if node har två barn:
//              hitta minsta värdet i höger delträd
//              ersätt node.item med detta värde
//              ta bort detta värde från höger delträd
//
//      return node
}
```

Basfall och rekursion

- **Basfall:** När noden är `null` → elementet finns inte
- **Rekursionssteg:** Traversera vänster eller höger tills rätt nod hittas

Hantering av krockar/dubbletter

I denna implementation hanteras dubbletter genom att de **ignoreras** i metoden `add`. Detta innebär att trädet endast innehåller unika värden.

Detta val gör implementationen enklare och påverkar även metoden `remove`, eftersom det aldrig finns flera noder med samma värde att ta bort.

Andra möjliga strategier hade varit:

- att lagra en räknare i noden
- att lägga dubbletter till vänster eller höger
- att kasta ett undantag

Att ignorera dubbletter är dock en vanlig lösning när man vill representera en mängd (set).

Kommentar om toString()

Metoden `toString()` implementerades för att skriva ut trädets innehåll med en **in-order traversal**, vilket ger en sorterad lista av elementen.

Denna metod finns inte i interfacet `Tree<T>` eftersom den inte är nödvändig för själva datastrukturens funktionalitet, utan används främst för testning och utskrift.

Uppgift 3 – Tidskomplexitet

Resultat – slumpade heltal

n	Tid T(n) (ms)	Tid T(2n) (ms)	Kvot T(2n)/T(n)
20 000	44	29	0,66
40 000	29	39	1,34
80 000	39	54	1,38
160 000	54	–	–

Resultat – sorterade heltal

n	Tid T(n) (ms)	Tid T(2n) (ms)	Kvot T(2n)/T(n)
5 000	947	1985	2,10
10 000	1985	2917	1,47
15 000	2917	–	–
20 000	StackOverflowError	–	–

Analys

Resultaten visar en tydlig skillnad i exekveringstid mellan slumpade och sorterade indata.

Slumpade heltal

För slumpade heltal ligger kvoterna runt cirka **1.3**, vilket innebär att exekveringstiden ökar relativt långsamt när antalet element fördubblas. Detta tyder på att trädet är relativt balanserat.

I ett balanserat binärt sökträd är tidskomplexiteten för sökning:

O(log n)

Resultaten stämmer därför relativt väl överens med den teoretiska tidskomplexiteten, eftersom tiden inte ökar drastiskt när n ökar.

Sorterade heltal

För sorterade heltal ser vi en helt annan utveckling. Tiderna ökar mycket snabbare och kvoten är i början över **2**, vilket tyder på en mer linjär tillväxt.

Dessutom uppstår ett **StackOverflowError** vid större storlek (n = 20000). Detta beror på att trädet blir kraftigt obalanserat och liknar en länkad lista. Då blir rekursionen i `add`-metoden mycket djup, vilket leder till att programmet kraschar.

I detta fall är tidskomplexiteten:

O(n)

Jämförelse med teori

Teoretiskt gäller:

- Balanserat BST → **O(log n)**
- Obalanserat BST → **O(n)**

Resultaten bekräftar detta tydligt:

- Slumpade data → bättre balans → snabbare sökning
- Sorterade data → obalanserat träd → långsammare sökning + kraschar

Slutsats

Experimentet visar att ett binärt sökträds prestanda beror starkt på hur data läggs in. Vid ogynnsam inmatning (sorterade värden) kan trädet bli mycket ineffektivt.

För att undvika detta används i praktiken ofta självbalanserande träd, såsom AVL-träd eller röd-svarta träd, som automatiskt håller trädet balanserat.